# Origin Validation

Origin validation is a security check where a WebSocket server verifies that a connection request is coming from an approved website or domain. It's like a bouncer at an exclusive party checking a guest's invitation to see if they came from an approved address.

---

## Why It's a Critical Security Measure

The primary reason to implement origin validation is to prevent a type of attack called `Cross-Site WebSocket Hijacking (CSWH)`.

Here's how a CSWH attack works without origin validation:

1. **You're Logged In:** You are logged into a legitimate website, say `my-bank.com`, which uses WebSockets for real-time notifications.
2. **You Visit a Malicious Site:** You then open another browser tab and visit a malicious website, `evil-hacker-site.com`.
3. **The Hijack:** The JavaScript running on `evil-hacker-site.com` can attempt to open a WebSocket connection directly to your bank's server (`ws://my-bank.com`).
4. **Automatic Authentication:** When this connection is attempted, your browser will automatically attach your authentication cookies for `my-bank.com` to the request. The server sees these valid cookies and thinks the connection request is from you.
5. **Data Theft:** The malicious script on `evil-hacker-site.com` now has a fully authenticated WebSocket connection to your bank's server. It can send commands and receive your private financial data without your knowledge or consent.

### How Origin Validation Prevents This

When the browser on `evil-hacker-site.com` tries to connect, it sends an `Origin` header in the request, telling the server where it's coming from: `Origin: https://evil-hacker-site.com`.

A secure server will perform origin validation:

1. It inspects the `Origin` header.
2. It checks it against a "whitelist" of approved domains (e.g., `https://my-bank.com`).
3. Since `https://evil-hacker-site.com` is **not** on the whitelist, the server **rejects the connection**.

The attack is stopped before it can even start. In short, origin validation is an essential security measure that ensures your server only talks to your own frontend application, preventing other websites from impersonating your users.